

✨ Zenith Framework v1.0.0 - Complete Guide

Table of Contents

1. [Introduction](#)
 2. [Installation](#)
 3. [Quick Start](#)
 4. [Core Concepts](#)
 5. [Database & ORM](#)
 6. [Routing](#)
 7. [Templates](#)
 8. [Authentication](#)
 9. [REST API](#)
 10. [Admin Interface](#)
 11. [Deployment](#)
 12. [Advanced Features](#)
-

Introduction

Zenith Framework is a revolutionary single-file Python web framework that combines the best features of Flask, FastAPI, Django, and more - all with **zero external dependencies**.

Key Features

- ✅ **Single File** - Entire framework in one `zenith.py` file
 - ✅ **Zero Dependencies** - Pure Python 3.8+ stdlib only
 - ✅ **Hybrid Server** - WSGI + threaded support
 - ✅ **Advanced ORM** - SQL models + NoSQL document store
 - ✅ **Auto-Admin** - Generated CRUD interface
 - ✅ **Auto-REST-API** - Automatic REST endpoints
 - ✅ **Security** - CSRF, sessions, password hashing
 - ✅ **cPanel Ready** - One-command deployment
 - ✅ **Educational** - Perfect for learning & teaching
 - ✅ **Production Ready** - Connection pooling, thread-safe
-

Installation

No installation needed! Just download `zenith.py`:

```
bash

# Download the framework
curl -O https://yourdomain.com/zenith.py

# Or create your app
touch myapp.py
```

Quick Start

Minimal Application

```
python

from zenith import Zenith, Model, String

app = Zenith(__name__)

@app.route('/')
def hello(req):
    return "Hello, Zenith!"

if __name__ == '__main__':
    app.run(debug=True)
```

Run it:

```
bash

python myapp.py
# Visit http://127.0.0.1:8080
```

With Database

```
python
```

```
from zenith import Zenith, Model, String, Integer

app = Zenith(__name__)

class User(Model):
    name = String(max_length=100)
    email = String(max_length=100, unique=True)
    age = Integer()

@app.route('/users')
def users(req):
    all_users = User.all()
    return app.render('users.html', users=all_users)

@app.post('/users')
def create_user(req):
    User.create(
        name=req.form['name'],
        email=req.form['email'],
        age=int(req.form['age'])
    )
    return app.redirect('/users')

if __name__ == '__main__':
    app.run(debug=True)
```

Core Concepts

Application Instance

```
python

from zenith import Zenith

# Create app with custom config
app = Zenith(__name__,
    SECRET_KEY='your-secret-key',
    DATABASE='myapp.db',
    DEBUG=True
)
```

Configuration

python

Default configuration

```
CONFIG = {  
    'SECRET_KEY': 'auto-generated',  
    'DATABASE': 'zenith.db',  
    'STATIC_FOLDER': 'static',  
    'TEMPLATE_FOLDER': 'templates',  
    'UPLOAD_FOLDER': 'uploads',  
    'DEBUG': False,  
    'HOST': '127.0.0.1',  
    'PORT': 8080,  
    'SESSION_LIFETIME': 3600,  
    'CSRF_ENABLED': True,  
    'POOL_SIZE': 5,  
}
```

Override in app

```
app = Zenith(__name__, PORT=5000, DEBUG=True)
```

Database & ORM

SQL Models

python

```
from zenith import Model, String, Integer, Float, Boolean, DateTime, Text, JSON, ForeignKey
```

```
class User(Model):  
    username = String(max_length=50, unique=True, nullable=False)  
    email = String(max_length=100, unique=True)  
    age = Integer(default=0)  
    is_active = Boolean(default=True)  
    bio = Text()  
    settings = JSON()
```

```
class Post(Model):  
    title = String(max_length=200)  
    content = Text()  
    author_id = ForeignKey(User)
```

CRUD Operations

```
python
```

Create

```
user = User.create(username='john', email='john@example.com', age=25)
```

Read

```
user = User.get(id=1)
```

```
user = User.get(username='john')
```

```
all_users = User.all()
```

Query with filters

```
adults = User.where(age__gte=18).all()
```

```
johns = User.where(username__contains='john').all()
```

Q Expressions (Advanced)

```
from zenith import Q
```

```
users = User.where(Q(age__gte=18) & Q(is_active=True)).all()
```

```
users = User.where(Q(age__lt=18) | Q(age__gt=65)).all()
```

Update

```
user.age = 26
```

```
user.save()
```

Delete

```
user.delete()
```

Chaining

```
users = User.where(is_active=True).order_by('-age').limit(10).all()
```

```
count = User.where(age__gte=18).count()
```

NoSQL Documents (Pods)

```
python
```

```
from zenith import Pod

class Article(Pod):
    """Dynamic document with any fields"""
    pass

# Create
article = Article.create(
    title='My Article',
    content='Content here',
    tags=['python', 'web'],
    metadata={'views': 0}
)

# Find
articles = Article.find(tags='python')
article = Article.get(id=1)

# Update
article.metadata['views'] += 1
article.save()
```

Routing

Basic Routes

```
python
```

```
@app.route('/')
def index(req):
    return "Home Page"

@app.route('/about')
def about(req):
    return app.render('about.html')

# HTTP method shortcuts
@app.get('/posts')
def list_posts(req):
    return Post.all()

@app.post('/posts')
def create_post(req):
    Post.create(**req.form)
    return app.redirect('/posts')

@app.put('/posts/<id>')
def update_post(req, id):
    post = Post.get(id=id)
    post.title = req.form['title']
    post.save()
    return "Updated"

@app.delete('/posts/<id>')
def delete_post(req, id):
    Post.get(id=id).delete()
    return "Deleted"
```

URL Parameters

python


```
@app.route('/user/<id>')
def user_profile(req, id):
    user = User.get(id=id)
    return app.render('profile.html', user=user)
```

```
@app.route('/posts/<int:year>/<int:month>')
def posts_by_date(req, year, month):
    # id is automatically converted to int
    return f"Posts from {month}/{year}"
```

Path parameter (captures slashes)

```
@app.route('/files/<path:filepath>')
def serve_file(req, filepath):
    return app.send_file(filepath)
```

Request Object

python

```
@app.post('/form')
def handle_form(req):
    # Form data
    name = req.form.get('name')
    email = req.form['email'] # Raises KeyError if missing

    # Query parameters
    page = req.args.get('page', 1)

    # JSON data
    data = req.json

    # Headers
    auth = req.headers.get('Authorization')

    # Cookies
    session_id = req.cookies.get('session_id')

    # Session
    req.session['user_id'] = 123

    # Method & Path
    if req.method == 'POST':
        print(f"Posted to {req.path}")

    return "OK"
```

Response Types

python

```
# String response
@app.route('/text')
def text_response(req):
    return "Plain text"

# HTML response
@app.route('/html')
def html_response(req):
    return app.render('template.html', data=...)

# JSON response
@app.route('/json')
def json_response(req):
    return {'status': 'ok', 'data': [...]}

# Custom response
from zenith import Response

@app.route('/custom')
def custom_response(req):
    resp = Response('Content', status=201)
    resp.headers['X-Custom'] = 'Value'
    resp.set_cookie('name', 'value')
    return resp

# Redirect
@app.route('/old')
def old_route(req):
    return app.redirect('/new')
```

Templates

Template Syntax

Create `templates/index.html`:

```
html
```

```
<!DOCTYPE html>
<html>
<head>
  <title>{{ title }}</title>
</head>
<body>
  <h1>{{ heading }}</h1>

  {% if user %}
    <p>Welcome, {{ user.name }}!</p>
  {% else %}
    <p>Please log in</p>
  {% endif %}

  <ul>
    {% for post in posts %}
      <li>{{ post.title }} by {{ post.author }}</li>
    {% endfor %}
  </ul>
</body>
</html>
```

Template Inheritance

templates/base.html:

```
html
```

```
<!DOCTYPE html>
<html>
<head>
  <title>{% block title %}Default Title{% endblock %}</title>
</head>
<body>
  <header>
    <nav>Site Navigation</nav>
  </header>

  <main>
    {% block content %} {% endblock %}
  </main>

  <footer>© 2025</footer>
</body>
</html>
```

templates/page.html:

```
html

{% extends "base.html" %}

{% block title %}My Page{% endblock %}

{% block content %}
  <h1>Page Content</h1>
  <p>This extends the base template</p>
{% endblock %}
```

Template Includes

```
html

{% include "partials/header.html" %}

<main>Content</main>

{% include "partials/footer.html" %}
```

Using Templates

```
python

@app.route('/page')
def page(req):
    return app.render('page.html',
        title='My Page',
        user=req.session.get('user'),
        posts=Post.all()
    )
```

Authentication

User Model

```
python

from zenith import Model, String, hash_password, verify_password

class User(Model):
    username = String(max_length=50, unique=True)
    email = String(max_length=100, unique=True)
    password_hash = String(max_length=200)

    @classmethod
    def create_user(cls, username, email, password):
        return cls.create(
            username=username,
            email=email,
            password_hash=hash_password(password)
        )

    def check_password(self, password):
        return verify_password(password, self.password_hash)
```

Login/Logout

```
python
```

```

from zenith import login_required

@app.post('/login')
def login(req):
    username = req.form['username']
    password = req.form['password']

    user = User.get(username=username)
    if user and user.check_password(password):
        req.session['user_id'] = user.id
        req.session['username'] = user.username
        return app.redirect('/dashboard')

    return app.render('login.html', error='Invalid credentials')

@app.route('/logout')
def logout(req):
    req.session.clear()
    return app.redirect('/')

@app.route('/dashboard')
@login_required
def dashboard(req):
    user_id = req.session['user_id']
    user = User.get(id=user_id)
    return app.render('dashboard.html', user=user)

```

Registration

```

python

@app.post('/register')
def register(req):
    User.create_user(
        username=req.form['username'],
        email=req.form['email'],
        password=req.form['password']
    )
    return app.redirect('/login')

```

REST API

Auto-Generated API

```
python

from zenith import create_rest_api

# Automatically creates:
# GET /api/users - List all users
# POST /api/users - Create user
# GET /api/users/<id> - Get user
# PUT /api/users/<id> - Update user
# DELETE /api/users/<id> - Delete user

create_rest_api(app, User)
```

Custom API Routes

```
python

@app.api('/api/search', methods=['GET'])
def search_api(req):
    query = req.args.get('q')
    results = User.where(username__contains=query).all()
    return [u.to_dict() for u in results]

@app.api('/api/stats', methods=['GET'])
def stats_api(req):
    return {
        'total_users': User.where().count(),
        'active_users': User.where(is_active=True).count()
    }
```

Admin Interface

Auto-Generated Admin

```
python
```



```
from zenith import create_admin, login_required
```

```
# Create admin for your models
```

```
create_admin(app, [User, Post, Article],  
             url_prefix='/admin',  
             auth_func=login_required)
```

Access at <http://localhost:8080/admin>

Features:

- List all records
 - Create new records
 - Edit existing records
 - Delete records
 - Automatic form generation
-

Deployment

Development Server

```
python  
  
if __name__ == '__main__':  
    app.run(  
        host='0.0.0.0', # Listen on all interfaces  
        port=8080,  
        debug=True,    # Auto-reload on changes  
        threaded=True  # Handle multiple requests  
    )
```

cPanel/Namecheap Deployment

```
python
```

```
# 1. Create passenger_wsgi.py
from zenith import create_passenger_wsgi
create_passenger_wsgi('myapp', 'app')

# 2. Upload files to public_html/:
# - myapp.py
# - zenith.py
# - passenger_wsgi.py
# - templates/
# - static/

# 3. In cPanel:
# - Setup Python App
# - Set Python version (3.8+)
# - Set startup file: passenger_wsgi.py
# - Click "Start"

# Done! Your app is live at yourdomain.com
```

Production WSGI

```
python

# app.py
from zenith import Zenith
app = Zenith(__name__, DEBUG=False)

# ... your routes ...

# For Gunicorn, uWSGI, etc.
application = app.wsgi

# Run with Gunicorn:
# gunicorn app:application -w 4
```

Advanced Features

Middleware

```
python
```

```
@app.before
def log_requests(req):
    print(f'{req.method} {req.path}')

@app.after
def add_header(req, response):
    response.headers['X-Powered-By'] = 'Zenith'
    return response
```

Error Handlers

```
python

@app.errorhandler(404)
def not_found(req, error):
    return app.render('404.html'), 404

@app.errorhandler(500)
def server_error(req, error):
    return "Something went wrong!", 500
```

CLI Commands

```
python

@app.cli('seed')
def seed_database():
    """Seed the database with test data"""
    User.create(username='admin', email='admin@example.com')
    print("Database seeded!")

# Run: python myapp.py seed
```

File Uploads

```
python
```

```
@app.post('/upload')
def upload_file(req):
    file = req.files['document']
    filename = secure_filename(file.filename)
    filepath = Path(app.config['UPLOAD_FOLDER']) / filename
    file.save(filepath)
    return "Uploaded!"
```

Flash Messages

```
python

@app.post('/action')
def some_action(req):
    # Do something...
    app.flash('Action completed successfully!', 'success')
    return app.redirect('/dashboard')

# In template:
# {% for msg in get_flashed_messages() %}
#     <div class="alert">{{ msg }}</div>
# {% endfor %}
```

URL Generation

```
python

@app.route('/user/<id>')
def user_profile(req, id):
    url = app.url_for('user_profile', id=123)
    # url = '/user/123'
    return app.render('profile.html', edit_url=url)
```

Complete Example Application

```
python
```

```

from zenith import (
    Zenith, Model, String, Integer, Text,
    create_rest_api, create_admin, login_required,
    hash_password, verify_password
)

app = Zenith(__name__, DEBUG=True)

# Models
class User(Model):
    username = String(max_length=50, unique=True)
    email = String(max_length=100)
    password_hash = String(max_length=200)

class Post(Model):
    title = String(max_length=200)
    content = Text()
    author_id = Integer()

# Routes
@app.route('/')
def index(req):
    posts = Post.all()
    return app.render('index.html', posts=posts)

@app.route('/login', methods=['GET', 'POST'])
def login(req):
    if req.method == 'POST':
        user = User.get(username=req.form['username'])
        if user and verify_password(req.form['password'], user.password_hash):
            req.session['user_id'] = user.id
            return app.redirect('/dashboard')
    return app.render('login.html')

@app.route('/dashboard')
@login_required
def dashboard(req):
    user = User.get(id=req.session['user_id'])
    posts = Post.where(author_id=user.id).all()
    return app.render('dashboard.html', user=user, posts=posts)

@app.post('/posts')
@login_required

```

```
def create_post(req):
    Post.create(
        title=req.form['title'],
        content=req.form['content'],
        author_id=req.session['user_id']
    )
    return app.redirect('/dashboard')

# Auto-generate REST API
create_rest_api(app, User)
create_rest_api(app, Post)

# Auto-generate Admin
create_admin(app, [User, Post])

if __name__ == '__main__':
    app.run()
```

Performance Tips

1. **Connection Pooling:** Automatically handled
2. **Threaded Server:** Enabled by default
3. **Template Caching:** Automatic in production
4. **Static File Caching:** Use CDN in production
5. **Database Indexes:** Add `index=True` to fields

```
python

class User(Model):
    email = String(max_length=100, unique=True, index=True)
```

Security Best Practices

1. **Always use HTTPS in production**
2. **Never commit SECRET_KEY to git**
3. **Use environment variables for secrets**

- 4. **Enable CSRF protection (default)**
- 5. **Hash passwords (use built-in functions)**
- 6. **Validate user input**
- 7. **Use prepared statements (automatic in ORM)**

Comparison with Other Frameworks

Feature	Zenith	Flask	FastAPI	Django
Dependencies	0	3+	10+	Many
File Count	1	Many	Many	Many
Auto Admin	✓	✗	✗	✓
Auto REST API	✓	✗	✓	With DRF
ORM Built-in	✓	✗	✗	✓
NoSQL Support	✓	✗	✗	✗
Async Support	⌚	✗	✓	✓
Learning Curve	Low	Low	Medium	High
cPanel Deploy	✓	✓	⚠	⚠

Support & Community

- **GitHub:** <https://github.com/yourusername/zenith>
- **Documentation:** <https://zenith-framework.readthedocs.io>
- **Discord:** <https://discord.gg/zenith>
- **Email:** support@zenith-framework.com

License

MIT License - See LICENSE file for details

Credits

Created with ❤️ by combining the best ideas from:

- Flask (routing simplicity)
- FastAPI (modern features)
- Django (batteries included)
- Fiole (minimalism)
- BlackBean (RedBean ORM style)
- Chatu (educational focus)

Made for developers, by developers. ✨